

Python API usage instructions

API (Application Programming Interface), also known as Application Programming Interface functions, are predefined functions. When using the following function interfaces, please import our API library at the beginning by entering the following code, otherwise it will not run successfully:

```
# Example
from pymycobot import MyCobot320

mc = MyCobot320('/dev/ttyAMA0', 115200)

# Gets the current angle of all joints
angles = mc.get_angles()
print(angles)

# Set 1 joint to move to 40 and speed to 20
mc.send_angle(1, 40, 20)
```

1. System Status

1.1 get_system_version()

- **function:** Get the machine master control version (pico firmware version)
- **Return value:** Firmware version number

1.2 get_atom_version()

- **function:** Get the atom version on the machine end
- **Return value:** Firmware version number

1.3 get_basic_version()

- **function:** Get basic firmware version for M5 version
- **Return value:** float firmware version

2. Overall Status

2.1 power_on()

- **function:** atom open communication (default open)
- **Return value:**
 - 1: complete

2.2 power_off()

- **function:** Power off of the robotic arm
- **Return value:**
 - 1: complete

2.3 is_power_on()

- **function:** judge whether robot arms is powered on or not
- **Return value:**
 - 1: power on
 - 0: power off
 - -1: error

2.4 release_all_servos(data=None)

- function:

release all robot arms

- Attentions: After the joint is disabled, it needs to be enabled to control within 1 second
- **Parameters:** data (optional) : The way to relax the joints. The default is damping mode, and if the 'data' parameter is provided, it can be specified as non damping mode (1-Undamping).
- Return value:
 - 1: complete

2.5 focus_servos(servo_id)

- **function:** Power on designated servo
- **Parameters:**
 - servo_id: int, 1-6
- Return value:
 - 1: complete

2.6 is_controller_connected()

- **function:** Wether connected with Atom
- **Return value:**
 - 1: succeed
 - 0: failed
 - -1: error data

2.7 read_next_error()

- **function:** Robot Error Detection
- **Return value:** list len 6
 - 0: No abnormality
 - 1: Communication disconnected
 - 2: Unstable communication
 - 3: Servo abnormality

2.8 `get_fresh_mode()`

- **function:** Query sports mode
- **Return value:**
 - `0`: Interpolation mode
 - `1`: Refresh mode

2.9 `set_fresh_mode()`

- **function:** Set command refresh mode
- **Parameters:**
 - `1`: Always execute the latest command first.
 - `0`: Execute instructions sequentially in the form of a queue.
- **Return value:**
 - `1`: complete

2.10 `get_robot_status()`

- **Function:** Get the robot self-check status to see if important parameters are normal.
- **Abnormal description:**
 - `0`: Communication abnormality, please check whether the line, servo firmware version is normal, whether the power is plugged in, whether the firmware is burned correctly, whether the baud rate is correct, etc.
 - `1`: The servo motor model is wrong and the motor needs to be replaced
 - `2`: The servo motor firmware version is low and needs to be upgraded using FD
 - `3`: The servo motor p value is abnormal, the default is 32, this abnormality will be automatically restored
 - `4`: The servo motor D value is abnormal, the default is 8, this abnormality will be automatically restored
 - `5`: The servo motor I value is abnormal, the default is 0, this abnormality will be automatically restored
 - `6`: The servo motor clockwise insensitive zone parameter is abnormal, the default is 3, this abnormality will be automatically restored
 - `7`: The servo motor counterclockwise insensitive zone parameter is abnormal, the default is 3, this abnormality will be automatically restored
 - `8`: The servo motor phase is abnormal, this abnormality will be automatically restored
 - `9`: The servo motor return delay is abnormal, the default is 0, this abnormality will be automatically restored
 - `10`: The servo motor minimum starting force is abnormal, the default is 0, this abnormality will be automatically restored
 - `11`: The servo motor is abnormal. When the servo is abnormal, the machine cannot be controlled. Please query the servo feedback interface `get_servo_status` to check the specific error.
 - `255`: Unknown error

2.11 `focus_all servos()`

- **Function:** All servos are powered on
- **Return value:**
 - `1`: complete

2.12 `set_vision_mode()`

- **Function:** Set the vision tracking mode, limit the attitude flip of `send_coords` in refresh mode. (Applicable only to vision tracking function)
- **Parameter:**
 - `1`: open
 - `0`: close
- **Return value:**
 - `1`: complete

3.MDI Mode and Operation

3.1 `get_angles()`

- **function:** get the degree of all joints
- **Return value:** `list` a float list of all degree

3.2 `send_angle(id, degree, speed)`

- **function:** send one degree of joint to robot arm
- **Parameters:**
 - `id`: Joint id, range int 1-6
 - `degree`: degree value(`float`)

Joint Id	Range
1	-168 ~ 168
2	-135 ~ 135
3	-145 ~ 145
4	-148 ~ 184
5	-168 ~ 168
6	-180 ~ 180

- `speed`

: the speed and range of the robotic arm's movement 1~100

- **Return value:**
- `1`: complete

3.3 send_angles(angles, speed)

- **function:** Send all angles to all joints of the robotic arm
- Parameters:
 - `angles`: a list of degree value(`List[float]`), length 6
 - `speed`: (`int`) 1 ~ 100
- Return value:
 - `1`: complete

3.4 get_coords()

- **function:** Obtain robot arm coordinates from a base based coordinate system
- **Return value:** a float list of coord:[x, y, z, rx, ry, rz]

3.5 send_coord(id, coord, speed)

- **function:** send one coord to robot arm
- Parameters:
 - `id`: send one coord to robot arm, 1-6 corresponds to [x, y, z, rx, ry, rz]
 - `coord`: coord value(`float`)

Coord Id	Range
x	-350 ~ 350
y	-350 ~ 350
z	-41 ~ 523.9
rx	-180 ~ 180
ry	-180 ~ 180
rz	-180 ~ 180

- `speed`

:(

`int`

) 1-100

- **Return value:**
- `1`: complete

3.6 `send_coords(coords, speed, mode)`

- **function::** Send overall coordinates and posture to move the head of the robotic arm from its original point to your specified point
- Parameters:
 - `coords`: : a list of coords value `[x,y,z,rx,ry,rz]` ,length6
 - `speed (int)`: 1 ~ 100
 - `mode: (int)` 0 - angular, 1 - linear
- Return value:
 - `1`: complete

3.7 `pause()`

- **function:** Control the instruction to pause the core and stop all movement instructions
- Return value:
 - :
 - `1` - stopped
 - `0` - not stop
 - `-1` - error
- Return value:
 - `1`: complete

3.8 `sync_send_angles(angles, speed, timeout=15)`

- **function:** Send the angle in synchronous state and return when the target point is reached
- Parameters:
 - `angles`: a list of degree value(`List[float]`), length 6
 - `speed: (int)` 1 ~ 100
 - `timeout`: default 15 s
- Return value:
 - `1`: complete

3.9 `sync_send_coords(coords, speed, mode=0, timeout=15)`

- **function:** Send the coord in synchronous state and return when the target point is reached
- Parameters:
 - `coords`: a list of coord value(`List[float]`), length 6
 - `speed: (int)` 1 ~ 100
 - `mode: (int)` 0 - angular (default) , 1 - linear
 - `timeout`: default 15 s
- Return value:
 - `1`: complete

3.10 `get_angles_coords()`

- **function:** Get joint angles and coordinates
- **Return value:** A list with a length of 12. The first six digits are angle information, and the last six digits are coordinate information.

3.11 `is_paused()`

- **function:** Check if the program has paused the move command
- Return value:
 - `1` - paused
 - `0` - not paused
 - `-1` - error

3.12 `resume()`

- **function:** resume the robot movement and complete the previous command

3.13 `stop()`

- **function:** stop all movements of robot
- Return value:
 - :
 - `1` - stopped
 - `0` - not stop
 - `-1` - error

3.14 `is_in_position(data, flag)`

- **function** : judge whether in the position.
- Parameters:
 - data: Provide a set of data that can be angles or coordinate values. If the input angle length range is 6, and if the input coordinate value length range is 6
 - flag data type (value range 0 or 1)
 - `0`: angle
 - `1`: coord
- Return value:
 - :
 - `1` - true
 - `0` - false
 - `-1` - error

3.15 `is_moving()`

- **function:** judge whether the robot is moving
- Return value:
 - `1` moving
 - `0` not moving
 - `-1` error

3.16 `angles_to_coords(angles)`

- **Function** : Convert angles to coordinates.
- Parameters:
 - `angles: list` List of floating points for all angles.
- **Return value:** `list` List of floating points for all coordinates.

3.17 `solve_inv_kinematics(target_coords, current_angles)`

- **Function** : Convert coordinates to angles.
- Parameters:
 - `target_coords: list` List of floating points for all coordinates.
 - `current_angles: list` List of floating points for all angles, current angles of the robot
- **Return value:** `list` List of floating points for all angles.

4. JOG Mode and Operation

4.1 `jog_angle(joint_id, direction, speed)`

- **function:** jog control angle
- Parameters:
 - `joint_id`: Represents the joints of the robotic arm, represented by joint IDs ranging from 1 to 6
 - `direction(int)`: To control the direction of movement of the robotic arm, input `0` as negative value movement and input `1` as positive value movement
 - `speed`: 1 ~ 100
- Return value:
 - `1`: complete

4.2 `jog_coord(coord_id, direction, speed)`

- **function:** jog control coord.
- Parameters:
 - `coord_id: (int)` Coordinate range of the robotic arm: 1~6
 - `direction: (int)` To control the direction of machine arm movement, `0` - negative value movement, `1` - positive value movement

- `speed`: 1 ~ 100
- Return value:
 - `1`: complete

4.3 `jog_rpy(end_direction, direction, speed)`

- **function:** Rotate the end around a fixed axis in the base coordinate system
- Parameters:
 - `end_direction`: (`int`) Roll, Pitch, Yaw (1-3)
 - `direction`: (`int`) To control the direction of machine arm movement, `1` - forward rotation, `0` - reverse rotation
 - `speed`: (`int`) 1 ~ 100
- Return value:
 - `1`: complete

4.4 `jog_increment_angle(joint_id, increment, speed)`

- **Function:** Angle stepping, single joint angle increment control
- Parameter:
 - `joint_id`: 1-6
 - `increment`: Incremental movement based on the current position angle
 - `speed`: 1~100
- Return value:
 - `1`: Completed

4.5 `jog_increment_coord(id, increment, speed)`

- **Function:** Coordinate stepping, single coordinate increment control
- Parameter:
 - `id`: Coordinate axis 1-6
 - `increment`: Incremental movement based on the current position coordinate
 - `speed`: 1~100
- Return value:
 - `1`: Completed

4.6 `set_encoder(joint_id, encoder, speed)`

- **function:** Set a single joint rotation to the specified potential value
- **Parameters**
 - `joint_id`: (`int`) 1-6
 - `encoder`: 0 ~ 4096
 - `speed`: 1 ~ 100

- Return value:
 - `1`: complete

4.7 `get_encoder(joint_id)`

- **function:** Set a single joint rotation to the specified potential value
- **Parameters**
 - `joint_id`: (`int`) 1-6
- **Return value:** (`int`) Joint potential value

4.8 `set_encoders(encoders, speed)`

- **function:** Set the six joints of the manipulator to execute synchronously to the specified position.
- **Parameters**
 - `joint_id`: (`int`) 1-6
 - `encoder`: 0 ~ 4096
 - `speed`: 1 ~ 100
- Return value:
 - `1`: complete

4.9 `get_encoders()`

- **function:** Get the six joints of the manipulator.
- **Return value:** (`list`) the list of encoders

5. Running status and Settings

5.1 `get_joint_min_angle(joint_id)`

- **function:** Gets the minimum movement angle of the specified joint
- Parameters:
 - `joint_id` : Enter joint ID (range 1-6)
- **Return value:** `float` Angle value

5.2 `get_joint_max_angle(joint_id)`

- **function:** Gets the maximum movement angle of the specified joint
- Parameters:
 - `joint_id` : Enter joint ID (range 1-6)
- **Return value:** `float` Angle value

5.3 `set_joint_min(id, angle)`

- **function:** Set minimum joint angle limit
- Parameters:
 - `id` : Enter joint ID (range 1-6)

- `angle`: Refer to the limit information of the corresponding joint in the [send_angle\(\)](#) interface, which must not be less than the minimum value
- Return value:
 - `1`: complete

5.4 `set_joint_max(id, angle)`

- **function:** Set maximum joint angle limit
- Parameters:
 - `id`: Enter joint ID (range 1-6)
 - `angle`: Refer to the limit information of the corresponding joint in the [send_angle\(\)](#) interface, which must not be greater than the maximum value
 - **Return value:**
 - `1`: complete

6. Joint motor control

6.1 `is_servo_enable(servo_id)`

- **function:** Detecting joint connection status
- **Parameters:** `servo_id` 1-6
- Return value:
 - `1`: Connection successful
 - `0`: not connected
 - `-1`: error

6.2 `is_all_servo_enable()`

- **function:** Detect the status of all joint connections
- Return value:
 - `1`: Connection successful
 - `0`: not connected
 - `-1`: error

6.3 `set_servo_calibration(servo_id)`

- **function:** The current position of the calibration joint actuator is the angle zero point
- Parameters
 - :
 - `servo_id`: 1 - 6
- Return value:
 - `1`: complete

6.4 `release_servo(servo_id)`

- **function:** Set the specified joint torque output to turn off
- Parameters
 - :
 - `servo_id`: 1 ~ 6
- Return value:
 - `1`: release successful
 - `0`: release failed
 - `-1`: error

6.5 `focus_servo(servo_id)`

- **function:** Set the specified joint torque output to turn on
- **Parameters:** `servo_id`: 1 ~ 6
- Return value:
 - `1`: focus successful
 - `0`: focus failed
 - `-1`: error

6.6 `set_servo_data(servo_id, data_id, value, mode=None)`

- **function:** Set the data parameters of the specified address of the steering gear
- Parameters
 - :
 - `servo_id`: (`int`) joint id 1 - 6
 - `data_id`: (`int`) Data address
 - `value`: (`int`) 0 - 4096
 - `mode`: 0 - indicates that value is one byte(default), 1 - 1 represents a value of two bytes.

6.7 `get_servo_data(servo_id, data_id, mode=None)`

- **function:** Read the data parameter of the specified address of the steering gear.
- Parameters
 - :
 - `servo_id`: (`int`) joint id 1 - 6
 - `data_id`: (`int`) Data address
 - `mode`: 0 - indicates that value is one byte(default), 1 - 1 represents a value of two bytes.
- **Return value:** 0 ~ 4096

6.8 joint_brake(joint_id)

- **function:** Make it stop when the joint is in motion, and the buffer distance is positively related to the existing speed
- Parameters:
 - `joint_id:(int)` joint id 1 - 6
- Return value:
 - `1`: complete

7. Servo state value

7.1 get_servo_speeds()

- **function:** Get the movement speed of all joints
- **Return value:** A list unit step/s

7.2 get_servo_currents()

- **function:** Get joint current
- **Return value:** A list 0 ~ 3250 mA

7.3 get_servo_voltages()

- **function:** Get joint voltages
- **Return value:** A list volts < 24 V

7.4 get_servo_status()

- **function:** Get the movement status of all joints
- **Return value:** A list,[voltage, sensor, temperature, current, angle, overload], a value of `0` means no error, a value of `1` indicates an error
- **Abnormal description:**
 - 0: Servo motor undervoltage/overvoltage, check the voltage, if it is 0, you need to modify the servo parameters; if it is greater than the actual value, the heat sink may be damaged
 - 1: Servo motor magnetic encoding abnormality
 - 2: Servo motor overtemperature
 - 3: Servo motor overcurrent
 - 5: Servo motor overload

7.5 get_servo_temps()

- **function:** Get joint temperature
- **Return value:** A list unit °C

7.6 `set_void_compensate(mode)`

- **function:** Set void compensation mode
- Parameters
 - `mode`: (int) 0 - close, 1 - open
- Return value:
 - `1`: complete

8. Robotic arm end IO control

8.1 `set_color(r, g, b)`

- **function:** Set the color of the end light of the robotic arm
- **Parameters:**
 - `r` (int): 0 ~ 255
 - `g` (int): 0 ~ 255
 - `b` (int): 0 ~ 255
- Return value:
 - `1`: complete

8.2 `set_digital_output(pin_no, pin_signal)`

- **function:** set IO statue
- Parameters
 - `pin_no` (int): Pin number
 - `pin_signal` (int): 0 / 1
- Return value:
 - `1`: complete

8.3 `get_digital_input(pin_no)`

- **function:** read IO statue
- **Parameters:** `pin_no` (int)
- **Return value:** signal

8.4 `set_pin_mode(pin_no, pin_mode)`

- **function:** Set the state mode of the specified pin in atom.
- Parameters
 - `pin_no` (int): Pin number
 - `pin_mode` (int): 0 - input, 1 - output, 2 - input_pullup
- Return value:
 - `1`: complete

9. Robotic arm end gripper control

9.1 `set_gripper_state(flag, speed, _type_1=None)`

- **function:** Adaptive gripper enable
- **Parameters:**
 - `flag (int)`: 0 - open 1 - close, 254 - release
 - `speed (int)`: 1 ~ 100
 - `_type_1 (int)`:
 - 1 : Adaptive gripper (default state is 1)
 - 2 : A nimble hand with 5 fingers
 - 3 : Parallel gripper
 - 4 : Flexible gripper
- Return value:
 - 1: complete

9.2 `set_gripper_value(gripper_value, speed, gripper_type=None)`

- **function:** Set the gripper value
- **Parameters:**
 - `gripper_value (int)`: 0 ~ 100
 - `speed (int)`: 1 ~ 100
 - `gripper_type (int)`:
 - 1 : Adaptive gripper (default state is 1)
 - 2 : A nimble hand with 5 fingers
 - 3 : Parallel gripper
 - 4 : Flexible gripper
- Return value:
 - 1: complete

9.3 `set_gripper_calibration()`

- **function:** Set the current position of the gripper to zero
- Return value:
 - 1: complete

9.4 `init_electric_gripper()`

- **function:** Electric gripper initialization (it needs to be initialized once after inserting and removing the gripper)
- Return value:
 - 1: complete

9.5 set_electric_gripper(status)

- **function:** Set Electric Gripper Mode
- Parameters
 - :
 - `status`: 0 - open, 1 - close.
- Return value:
 - `1`: complete

9.6 set_gripper_mode(mode)

- **function:** Set gripper mode
- Parameters
 - :
 - `mode`: 0 - transparent transmission. 1 - Port Mode.
- Return value:
 - `1`: complete

9.7 get_gripper_mode()

- **function:** Get gripper mode
- Return value
 - :
 - `mode`: 0 - transparent transmission. 1 - Port Mode.

10. Set bottom IO input/output status

10.1 set_basic_output(pin_no, pin_signal)

- **function:** Set Base IO Output
- Parameters
 - :
 - `pin_no (int)` Pin port number
 - `pin_signal (int)`: 0 - low. 1 - high
- Return value:
 - `1`: complete

10.2 get_basic_input(pin_no)

- **function:** Read base IO input
- Parameters:
 - `pin_no (int)` pin number
- **Return value:** 0 - low. 1 - high

11. WLAN Setting

11.1 `set_ssid_pwd(account, password)`

- **function:** Change connected wifi. (Apply to m5)
- Parameters:
 - `account` (`str`) new wifi account
 - `password` (`str`) new wifi password
- Return value:
 - `1`: complete

11.2 `get_ssid_pwd()`

- **function:** Get connected wifi account and password. (Apply to m5)
- **Return value:** (account, password)

11.3 `set_server_port(port)`

- **function:** Change the connection port of the server
- Parameters:
 - `port` (`int`) The new connection port of the server.
- Return value:
 - `1`: complete

12. TOF

12.1 `get_tof_distance()`

- **function:** Get the detected distance (Requires external distance detector)
- **Return value:** (`int`) The unit is mm.

13. Cartesian space coordinate parameter setting

13.1 `set_tool_reference(coords)`

- **function:** Set tool coordinate system.
- Parameters:
 - `coords`: (`list`) [`x`, `y`, `z`, `rx`, `ry`, `rz`].
- Return value:
 - `1`: complete

13.2 `get_tool_reference(coords)`

- **function:** Get tool coordinate system.
- Return value:
 - `coords`: (`list`) [`x`, `y`, `z`, `rx`, `ry`, `rz`]

13.3 `set_world_reference(coords)`

- **function:** Set world coordinate system.
- Parameters:
 - `coords: (list) [x, y, z, rx, ry, rz]`.
- Return value:
 - `1`: complete

13.4 `get_world_reference()`

- **function:** Get world coordinate system.
- **Return value:** `(list) [x, y, z, rx, ry, rz]`.

13.5 `set_reference_frame(rftype)`

- **function:** Set base coordinate system.
- Parameters:
 - `rftype`: 0 - base 1 - tool.
- Return value:
 - `1`: complete

13.6 `get_reference_frame()`

- **function:** Get base coordinate system.
- **Return value:** `(list) [x, y, z, rx, ry, rz]`.

13.7 `set_movement_type(move_type)`

- **function:** Set movement type.
- Parameters:
 - `move_type`: 1 - moveI, 0 - moveJ.
- Return value:
 - `1`: complete

13.8 `get_movement_type()`

- **function:** Get movement type.
- Return value:
 - `1` - moveI
 - `0` - moveJ

13.9 `set_end_type(end)`

- **function:** Set end coordinate system
- Parameters:
 - `end (int)`: 0 - flange, 1 - tool
- Return value:
 - 1: complete

13.10 `get_end_type()`

- **function:** Obtain the end coordinate system
- Return value:
 - 0 - flange
 - 1 - tool

14. `utils (module)`

This module supports some helper methods. Use the code entered at the beginning of the file to import the module:

```
from pymycobot import utils
```

14.1 `utils.get_port_list()`

- **Function:** Get a list of all current serial port numbers
- **Return value:** Serial port list (`list`)

14.2 `utils.detect_port_of_basic()`

- **Function:** Return the first detected serial port number of M5 Basic. (Only one serial port number will be returned)
- **Return value:** Return the detected port number. If no serial port number is detected, it will return: None

15. Pro force-controlled gripper

15.1 `set_pro_gripper(gripper_id, address, value)`

- **Function:** Set the parameters of the Pro force-controlled gripper. You can set a variety of parameter functions. For details, please see the table below.
- Parameter:
 - :
 - `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
 - `address (int)`: The command number of the gripper.
 - `value`: The parameter value corresponding to the command number.

Function	gripper_id	address	value
Set gripper ID	14	3	1 ~ 254
Set gripper enable status	14	10	0 or 1, 0 - off enable; 1 - on enable
Set gripper clockwise runnable error	14	21	0 ~ 16
Set gripper counterclockwise runnable error	14	23	0 ~ 16
Set gripper minimum starting force	14	25	0 ~ 254
IO output settings	14	29	0, 1, 16, 17
Set IO opening angle	14	30	0 ~ 100
Set IO closing angle	14	31	0 ~ 100
Set servo virtual position value	14	41	0 ~ 100
Set clamping current	14	43	1 ~ 254

- Return value

:

- Please refer to the following table:

Function	Return
Set gripper ID	0 - Failure; 1 - Success
Set gripper enable status	0 - Failure; 1 - Success
Set gripper clockwise runnable error	0 - Failure; 1 - Success
Set gripper counterclockwise runnable error	0 - Failure; 1 - Success
Set gripper minimum starting force	0 - Failure; 1 - Success
IO output setting	0 - Failure; 1 - Success
Set IO opening angle	0 - Failure; 1 - Success
Set IO closing angle	0 - Failed; 1 - Success
Set servo virtual position value	0 - Failed; 1 - Success
Set holding current	0 - Failed; 1 - Success

15.2 `get_pro_gripper(gripper_id, address)`

- **Function:** Get the parameters of the Pro force-controlled gripper, and you can get a variety of parameter functions. For details, please see the table below.

- Parameter

:

- `gripper_id` (`int`): Gripper ID, default 14, value range 1 ~ 254.
- `address` (`int`): The command number of the gripper.

Function	<code>gripper_id</code>	<code>address</code>
Read firmware major version number	14	1
Read firmware minor version number	14	2
Read gripper ID	14	4
Read gripper clockwise runnable error	14	22
Read gripper counterclockwise runnable error	14	24
Read gripper minimum starting force	14	26
Read IO opening angle	14	34
Read IO closing angle	14	35
Get the amount of data in the current queue	14	40
Read servo virtual position value	14	42
Read the clamping current	14	44

- Return value

:

- See the following table (if the return value is -1, it means that no data can be read):

Function	Return
Read the firmware major version number	Major version number
Read the firmware minor version number	Minor version number
Read the gripper ID	1 ~ 254
Read the gripper clockwise runnable error	0 ~ 254
Read the gripper counterclockwise runnable error	0 ~ 254

Function	Return
Read the gripper minimum starting force	0 ~ 254
Read the IO opening angle	0 ~ 100
Read the IO closing angle	0 ~ 100
Get the amount of data in the current queue	Return the amount of data in the current absolute control queue
Read the servo virtual position value	0 ~ 100
Read the clamping current	1 ~ 254

15.3 `set_pro_gripper_angle(gripper_id, gripper_angle)`

- **Function:** Set the force-controlled gripper angle.
- Parameter
 - :
 - `gripper_id` (int): Gripper ID, default 14, value range 1 ~ 254.
 - `gripper_angle` (int): Gripper angle, value range 0 ~ 100.
- Return value
 - :
 - 0 - Failed
 - 1 - Success

15.4 `get_pro_gripper_angle(gripper_id)`

- **Function:** Read the angle of the force-controlled gripper.
- Parameter
 - :
 - `gripper_id` (int): Gripper ID, default 14, value range 1 ~ 254.
- **Return value:** int 0 ~ 100

15.5 `set_pro_gripper_open(gripper_id)`

- **Function:** Open the force-controlled gripper.
- Parameter
 - :
 - `gripper_id` (int): Gripper ID, default 14, value range 1 ~ 254.
- Return value
 - :
 - 0 - Failed
 - 1 - Success

15.6 set_pro_gripper_close(gripper_id)

- **Function:** Close the force-controlled gripper.
- Parameter
 - :
 - `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
- Return value
 - :
 - 0 - Failed
 - 1 - Success

15.7 set_pro_gripper_calibration(gripper_id)

- **Function:** Set the zero position of the force-controlled gripper. (The zero position needs to be set first when using it for the first time)
- Parameter
 - :
 - `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
- Return value
 - :
 - 0 - Failed
 - 1 - Success

15.8 get_pro_gripper_status(gripper_id)

- **Function:** Read the gripping status of the force-controlled gripper.
- Parameter
 - :
 - `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
- Return value:
 - 0 - Moving.
 - 1 - Stopped moving, no object was detected.
 - 2 - Stopped moving, object was detected.
 - 3 - After the object was detected, it fell.

15.9 set_pro_gripper_torque(gripper_id, torque_value)

- **Function:** Set the torque of the force-controlled gripper.
- Parameter
 - :
 - `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
 - `torque_value (int)`: Torque value, value range 100 ~ 300.
- Return value

- :
- 0 - Failed
- 1 - Success

15.10 `get_pro_gripper_torque(gripper_id)`

- **Function:** Read the torque of the force-controlled gripper.
- Parameter

- :
- `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
- **Return value:** (`int`) 100 ~ 300

15.11 `set_pro_gripper_speed(gripper_id, speed)`

- **Function:** Set the force-controlled gripper speed.
- Parameter

- :
- `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
- `speed (int)`: Gripper movement speed, value range 1 ~ 100.

- Return value

- :
- 0 - Failed
- 1 - Success

15.12 `get_pro_gripper_default_speed(gripper_id, speed)`

- **Function:** Read the default speed of the force-controlled gripper.
- Parameter

- :
- `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.

- **Return value:** Gripper default movement speed, range 1 ~ 100.

15.13 `set_pro_gripper_abs_angle(gripper_id, gripper_angle)`

- **Function:** Set the absolute angle of the force-controlled gripper.
- Parameter

- :
- `gripper_id (int)`: Gripper ID, default 14, value range 1 ~ 254.
- `gripper_angle (int)`: Gripper angle, value range 0 ~ 100.

- Return value

- :
- 0 - Failed
- 1 - Success

15.14 `set_pro_gripper_pause(gripper_id)`

- **Function:** Pause motion.
- Parameter
 - :
 - `gripper_id (int)` Gripper ID, default 14, value range 1 ~ 254.
- Return value
 - :
 - 0 - Failed
 - 1 - Success

15.15 `set_pro_gripper_resume(gripper_id)`

- **Function:** Motion recovery.
- Parameter
 - :
 - `gripper_id (int)` Gripper ID, default 14, value range 1 ~ 254.
- Return value
 - :
 - 0 - Failed
 - 1 - Success

15.16 `set_pro_gripper_stop(gripper_id)`

- **Function:** Stop motion.
- Parameter
 - :
 - `gripper_id (int)` Gripper ID, default 14, value range 1 ~ 254.
- Return value
 - :
 - 0 - Failed
 - 1 - Success

MyCobot 320 Socket

Note: The robotic arm that uses this class of premise has a server and has been turned on.

Use TCP/IP to control the robotic arm

1. Client

```
# demo
from pymycobot import MyCobot320Socket
# Port 9000 is used by default
mc = MyCobot320Socket("192.168.10.10", 9000)

res = mc.get_angles()
print(res)

mc.send_angles([0, 0, 0, 0, 0, 0], 20)
...
```

2. Server

Please see the [TCP/IP](#) section for details.